

Project Interim Report
On
File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming with Java

SESSION: 2025-26



Submitted By
Student name :
Raj Jaiswal
Reg No.11091

Bachelor of Technology in Computer Science &
Engineering with AI and ML

Under the Guidance of
Mr.Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING RUNGTA INTERNATIONAL SKILLS
UNIVERSITY BHILAI, CG

(April 2026)

Abstract

The Geometric Shape Area Calculator is an Object-Oriented Programming (OOP) based project designed to compute the area of different geometric shapes using a structured and modular approach. The project primarily focuses on implementing abstraction through interfaces and demonstrating polymorphism in Java. A common interface named *Shape* is defined, which declares a method for area calculation. Various classes such as *Circle* and *Square* implement this interface and provide their own specific logic to compute area based on mathematical formulas.

This project is needed to simplify and standardize the process of calculating areas of different shapes while maintaining code flexibility and reusability. Instead of writing separate logic for each shape, a unified structure allows easy extension by adding new shapes without modifying existing code.

The system works by creating objects of different shape classes and accessing them through a common interface reference. This demonstrates runtime polymorphism, where the method executed depends on the object type. Overall, the project highlights key OOP concepts and promotes clean, maintainable, and scalable software design.

Introduction

The Geometric Shape Area Calculator is a software application developed using Object-Oriented Programming (OOP) principles in Java. The main purpose of this project is to calculate the area of different geometric shapes in a structured, efficient, and reusable way. In real-world applications, calculating areas is a common requirement in fields such as engineering, architecture, and computer graphics. This project provides a simple and flexible solution to perform such calculations.

The system is designed using abstraction by defining a common interface called *Shape*, which declares a method for calculating area. Different classes such as *Circle* and *Square* implement this interface and provide their own specific logic based on mathematical formulas. This approach ensures that the code remains clean, modular, and easy to extend.

One of the key features of this project is polymorphism, where a single interface reference can be used to handle multiple types of shapes. This reduces code duplication and improves maintainability. Additionally, new shapes can be added easily without modifying existing code, making the system scalable.

Overall, this project demonstrates the practical implementation of core OOP concepts such as abstraction, method overriding, and polymorphism, while solving a real-world problem in a systematic way.

Scope

The Geometric Shape Area Calculator project is designed as a simple and efficient application for calculating the areas of basic geometric shapes such as *Circle* and *Square*. It is suitable for students, beginners in programming, and small-scale applications where quick and accurate area calculations are required. The project mainly focuses on demonstrating core Object-Oriented Programming concepts like abstraction, interfaces, and polymorphism.

The scope of this project can be extended significantly in the future. Additional shapes such as *Rectangle*, *Triangle*, and *Cylinder* can be easily added without modifying the existing code structure. The system can also be enhanced by integrating a graphical user interface (GUI) to make it more user-friendly. Furthermore, database integration can be implemented to store calculation history, making it useful for larger applications.

This project can also be adapted for use in educational tools, engineering calculations, and design

software. Overall, it provides a strong foundation for building more advanced and scalable applications using OOP principles.

System Requirements

1. Hardware Requirements

- Computer/Laptop
- Minimum **4 GB RAM** (8 GB recommended for better performance)
- Processor: Intel i3 or above
- Minimum 500 MB free disk space
- Keyboard and Monitor

2. Software Requirements

- **Operating System:** Windows / macOS / Linux
- **Programming Language:** Java
- **Java Development Kit (JDK):** JDK 8 or above
- **IDE (Integrated Development Environment):** VS Code / Eclipse / Edit Plus / IntelliJ IDEA
- **Compiler:** Java Compiler (comes with JDK)

3. Additional Requirements (Optional)

- Internet connection (for downloading tools and updates)
- Basic knowledge of Java programming and OOP concepts

Methodology

The Geometric Shape Area Calculator follows a structured approach based on Object-Oriented Programming principles. The working of the project is divided into the following steps:

1. Define Interface

First, a common interface named *Shape* is created. This interface declares a method `calculateArea()` which acts as a contract for all shape classes.

2. Create Shape Classes

Next, different classes such as *Circle* and *Square* are created. These classes implement the *Shape* interface and provide their own specific implementation of the `calculateArea()` method using appropriate mathematical formulas.

3. Object Creation

In the main method, objects of different shapes are created by passing required values (like radius for circle and side for square).

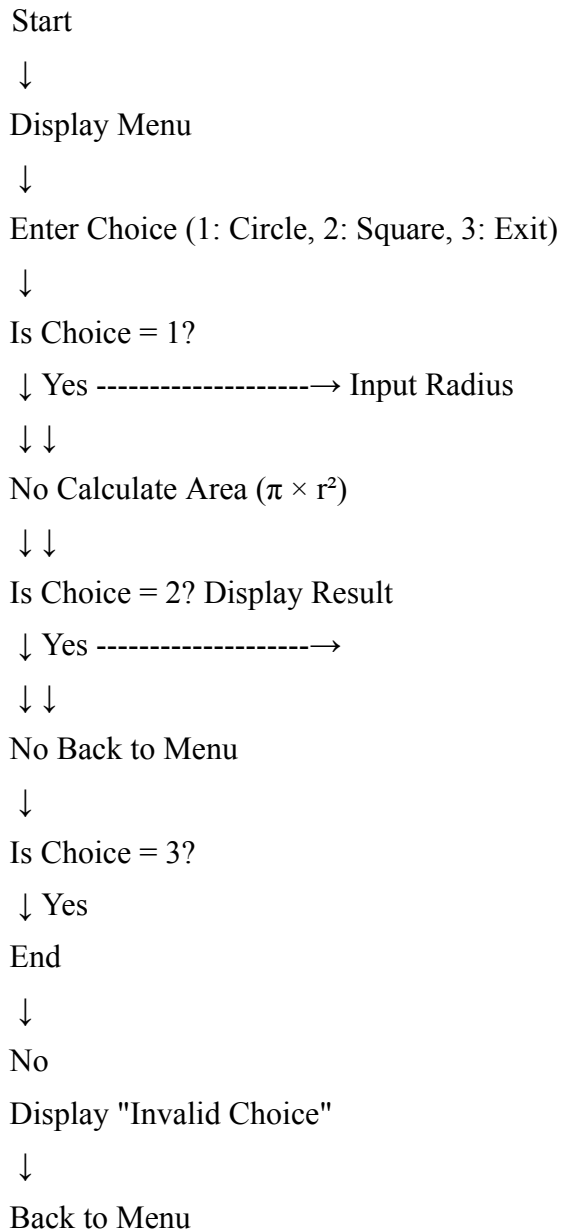
4. Apply Polymorphism

The objects are referenced using the *Shape* interface. This allows the same method `calculateArea()` to behave differently depending on the object type (runtime polymorphism).

5. Area Calculation

When the method is called, the program automatically executes the correct formula based on the shape type.

Flowchart



Class Diagram

Shape (Interface)

+ calculateArea(): double



||

Circle Square

- radius - side

+ calculateArea() + calculateArea()

Class name

Class: Shape (Interface)

Variables:

- None

Methods:

- calculateArea() : double

Array:

- Not present

Class: Circle

Variables:

- radius : double

Methods:

- Circle(double radius) (Constructor)
- calculateArea() : double **Array:**
- Not present

Class: Square

Variables:

- side : double

Methods:

- Square(double side) (Constructor)
- calculateArea() : double

Array:

- Not present

Class: Main

Variables:

- `shapes[] : Shape` (Array of Shape objects)

Methods:

- `main(String[] args)`

Array:

- Present $\rightarrow$ `Shape shapes[]`

Implementation

The Geometric Shape Area Calculator is implemented using Object-Oriented Programming principles such as abstraction, interfaces, and polymorphism. The program is divided into multiple classes, each responsible for a specific task.

First, a common interface named *Shape* is created. This interface defines a method for calculating the area. It does not contain any implementation but acts as a contract that ensures all shapes follow the same structure.

Next, concrete classes such as *Circle* and *Square* are created. Each class implements the *Shape* interface and provides its own logic for calculating the area. The Circle class uses the radius to calculate area using the formula $\pi \times \text{radius}^2$, while the Square class uses the side length to calculate area using $\text{side} \times \text{side}$. This shows how different classes can have different implementations of the same method.

In the Main class, objects of different shapes are created. These objects are referenced using the *Shape* interface, which allows the program to achieve polymorphism. When the area calculation method is called, the program decides at runtime which method implementation to execute based on the object type.

Finally, the calculated areas are displayed as output. This structured approach ensures the program is modular, reusable, and easy to extend by adding new shapes in the future.

Gantt chart

1.Planning & Structure

- **Requirement Analysis:** Understand project and OOP concepts
- **System Design:** Create interface and class structure
- **Coding:** Implement Circle and Square classes
- **Implementation:** Apply polymorphism in main class
- **Testing:** Check output and fix errors
- **Documentation:** Prepare report and diagrams
- **Submission:** Final review and submit project

2 .Time Managements

Time management played an important role in the successful completion of this project. The work was divided into small tasks and completed within a fixed schedule to ensure timely submission.

- **Planning Phase:** Time was allocated to understand the project requirements and OOP concepts.
- **Design Phase:** Proper time was given to design the interface and class structure.
- **Development Phase:** Coding was done step-by-step, focusing on one class at a time.
- **Testing Phase:** Sufficient time was used to test the program with different inputs and fix errors.
- **Documentation Phase:** Time was reserved to prepare the report, diagrams, and final presentation.

By following a structured schedule, the project was completed efficiently without delays. Proper time management helped in maintaining accuracy, improving productivity, and ensuring all project components were completed successfully.

Conclusion

The project successfully demonstrates the use of Object-Oriented Programming concepts in Java, particularly abstraction, interfaces, and polymorphism. By designing a Geometric Shape Area Calculator, a structured and modular system was developed to compute the areas of different shapes such as Circle and Square.

The use of a common interface ensures code reusability and flexibility, allowing new shapes to be added easily without modifying existing code. The implementation of polymorphism enables the program to handle multiple object types through a single interface, improving efficiency and maintainability.

Overall, the project provides a functional and extensible solution for area calculation while strengthening the understanding of core OOP principles. It also lays a strong foundation for developing more advanced applications in the future.

References

Book

Format:

[1] Author(s), *Book Title*, Edition. City: Publisher, Year.

Example:

[1] H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

Website

Format:

[2] Author (if any), "Title of page," Website Name. [Online]. Available: URL

Example:

[2] Oracle, "Java Documentation," Oracle. [Online]. Available:

Conference Paper

Format:

[3] Author(s), "Paper title," in Proc. Conference Name, year, pp.

Example:

[3] S. Kumar, "AI in Education," in Proc. IEEE Conf. on AI, 2021, pp. 120-125.

Journal Article

Format:

[4] Author(s), "Title of paper," Journal Name, vol., no., pp., year.

Example:

[4] A. Sharma and R. Gupta, "Machine Learning Techniques," International Journal of AI, vol. 5, no. 2, pp. 45-50, 2020.